

JobFlow 概要设计说明书

1. 系统概述

1.1 项目背景

JobFlow 是一款面向求职者的桌面端智能求职辅助系统，主要用于解决求职过程中岗位发现效率低、岗位筛选困难、公司信息分散以及投递记录难以管理等问题。

传统求职方式下，用户需要分别进入多个招聘平台，根据关键词搜索岗位，并逐条阅读岗位描述，再结合自身经历判断是否适合投递。当需要同时关注大量岗位时，用户需要投入大量时间进行重复的信息检索和人工筛选。同时，不同平台的岗位信息结构并不统一，相同岗位可能在多个平台重复出现，进一步增加了岗位筛选和管理成本。

JobFlow 将用户简历、求职偏好以及招聘平台岗位信息进行统一管理，并利用 Agent 对简历和岗位进行理解、匹配和分析，在此基础上为用户推荐更加符合个人条件的岗位。同时，系统提供公司背景信息分析、岗位投递以及投递状态跟踪等功能，从而形成完整的求职辅助流程。

1.2 系统目标

JobFlow 的总体目标是构建一个以用户求职画像为基础、以 Agent 为核心智能能力、以多平台岗位检索为主要数据来源的智能求职工作台。

系统主要实现以下目标：

- 将用户非结构化简历转换为结构化求职画像；
- 根据用户求职条件从多个招聘平台发现潜在岗位；
- 对不同来源岗位进行统一建模、清洗和去重；
- 对岗位与用户背景进行多维度匹配分析；
- 对岗位进行推荐排序并解释推荐原因；
- 聚合公开企业信息，辅助用户判断公司背景；
- 在用户确认的基础上辅助完成岗位投递；
- 对投递后的岗位进行统一状态管理和跟踪；
- 为后续简历优化和个性化求职策略提供扩展基础。

1.3 设计原则

系统概要设计遵循以下原则：

模块化原则。

按照业务职责对系统进行模块划分，降低模块之间的耦合程度。

Agent 与业务解耦原则。

Agent 负责复杂的信息理解和决策辅助，业务系统负责数据管理、任务调度和流程控制，避免将全部业务逻辑放入 Prompt 中。

平台适配原则。

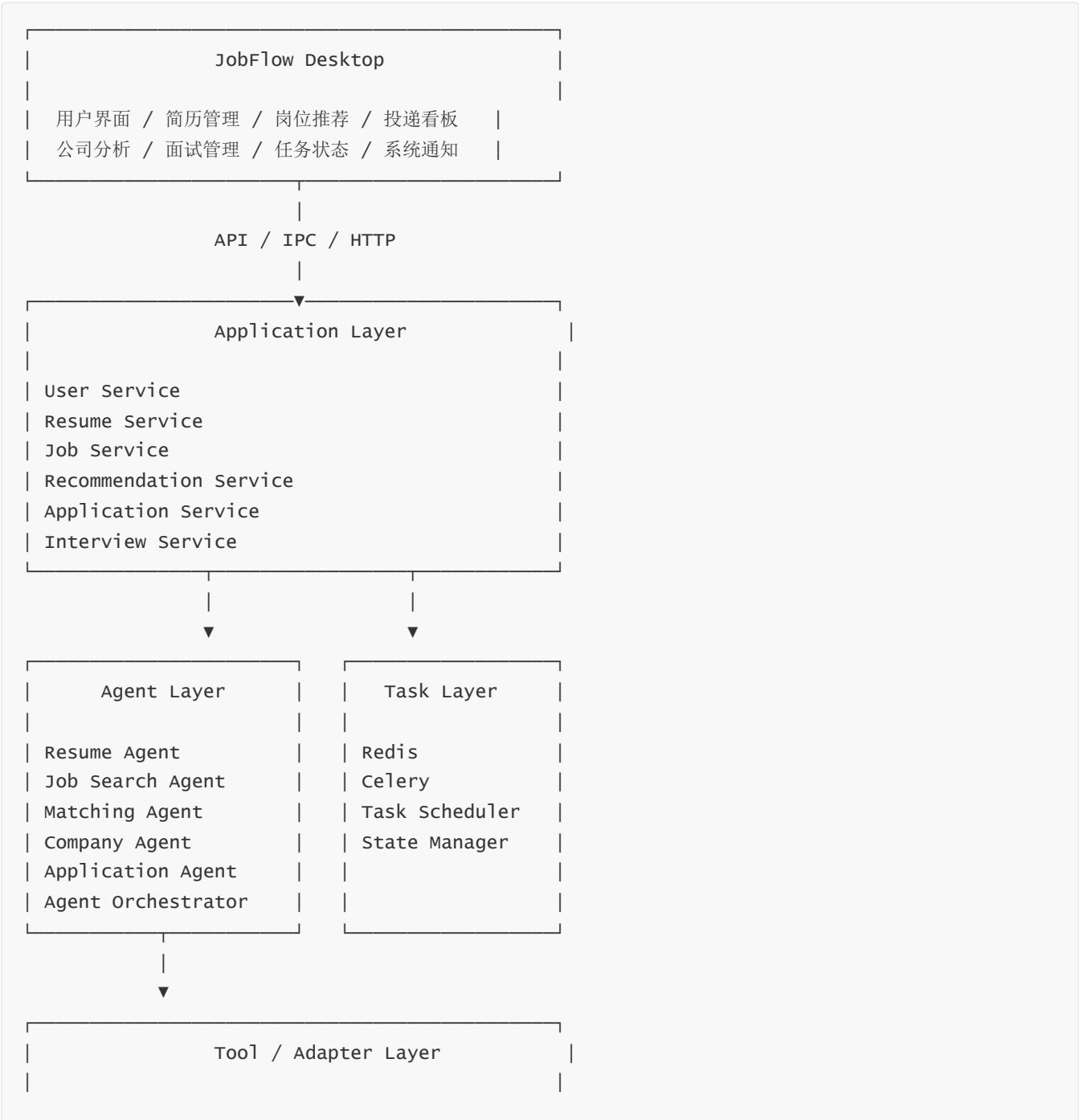
针对不同招聘平台建立独立的数据采集适配层，使新增招聘平台时无需修改核心业务逻辑。

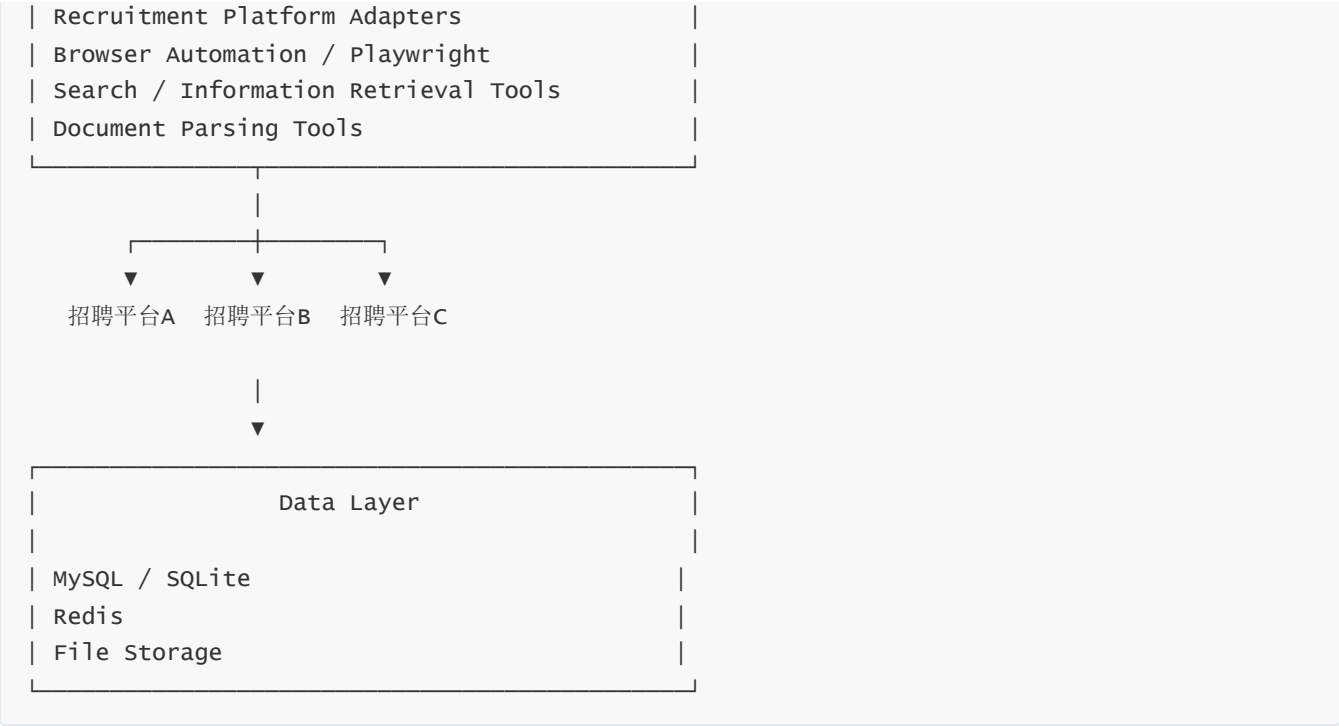
- 异步任务原则。**
对于岗位搜索、简历解析、公司调查和自动投递等耗时任务采用异步执行方式，避免阻塞桌面端 UI。
- 人工可接管原则。**
对于验证码、登录失效以及无法确定页面状态等自动化场景，系统允许任务暂停并交由用户处理。
- 数据安全原则。**
简历和联系方式等敏感数据应得到保护，尽可能采用本地优先的存储方式，并对敏感日志进行脱敏。

2. 系统总体架构

2.1 总体架构

JobFlow 采用桌面端 + 服务端 + Agent + 外部平台的分层架构。





其中，桌面端主要负责用户交互以及本地资源访问；应用服务层负责具体业务逻辑；Agent 层负责信息理解和智能分析；工具层负责连接招聘平台、浏览器和外部信息源；数据层负责持久化系统核心业务数据。

2.2 桌面端设计

JobFlow 采用桌面应用形式，前端主要负责：

- 用户界面展示；
- 简历文件选择；
- 求职偏好配置；
- 岗位推荐展示；
- 岗位详情查看；
- 公司信息展示；
- 投递看板；
- 面试日程；
- Agent 任务状态展示；
- 系统通知。

桌面端前端采用 Vue3 + Element Plus 实现。

桌面容器可以采用 Electron 或 Tauri。其具体选型可以在技术设计阶段进一步确定。

2.3 服务端设计

服务端采用 FastAPI 构建业务 API，负责：

- 用户数据管理；

- 简历数据管理；
- 岗位数据管理；
- 推荐结果管理；
- 投递记录管理；
- Agent 任务创建；
- 异步任务调度；
- 外部服务调用；
- 系统日志管理。

服务端不直接承担桌面端的全部任务，而是通过任务队列将耗时工作交由后台 Worker 执行。

3. 系统功能模块设计

3.1 用户画像模块

用户画像模块负责维护用户的基本信息、简历信息以及求职偏好。

主要功能包括：

- 简历上传；
- 简历解析；
- Candidate Profile 生成；
- 求职偏好设置；
- 用户信息修改。

该模块是整个系统的数据入口，为后续岗位搜索和匹配提供统一的用户画像。

3.2 简历解析模块

简历解析模块负责将 PDF、DOCX 或图片格式的简历转换为结构化信息。

处理流程：

```
graph TD; A[原始简历] --> B[文件解析]; B --> C[文本 / 图像内容提取]; C --> D[Resume Agent]; D --> E[结构化信息]; E --> F[Profile 校验]; F --> G[Candidate Profile];
```

模块输出主要包括：

基本信息

教育经历

工作经历

项目经历

技能

证书

获奖情况

其他经历

解析结果需要经过格式校验，以减少 LLM 输出异常对系统后续业务造成的影响。

3.3 岗位采集模块

岗位采集模块负责从多个招聘平台获取岗位数据。

为了适应不同平台之间的页面结构和字段差异，系统采用 Adapter 设计。

Job Source

|

├─ Platform Adapter A

├─ Platform Adapter B

├─ Platform Adapter C

└─ Other Adapter

每一个 Adapter 负责完成对应平台的：

- 岗位搜索；
- 岗位详情获取；
- 公司信息获取；
- 原始数据转换。

核心业务系统只接收标准化 Job 对象，而无需了解具体平台的页面结构。

3.4 岗位数据标准化模块

来自不同平台的岗位数据格式存在差异，需要在进入核心业务系统之前进行标准化。

标准化后的 Job 对象包括：

Job

├─ job_id

├─ title

├─ company_id

├─ city

├─ salary

├─ education

└─ experience

```
├─ job_type
├─ responsibilities
├─ requirements
├─ publish_time
├─ source
└─ source_url
```

系统同时保留部分原始岗位数据，以支持后续调试和数据追溯。

3.5 岗位去重模块

相同岗位可能在多个招聘平台重复发布，因此需要进行岗位去重。

系统可以综合：

- 公司名称；
- 岗位名称；
- 城市；
- 岗位描述；
- 招聘要求；
- 发布时间；

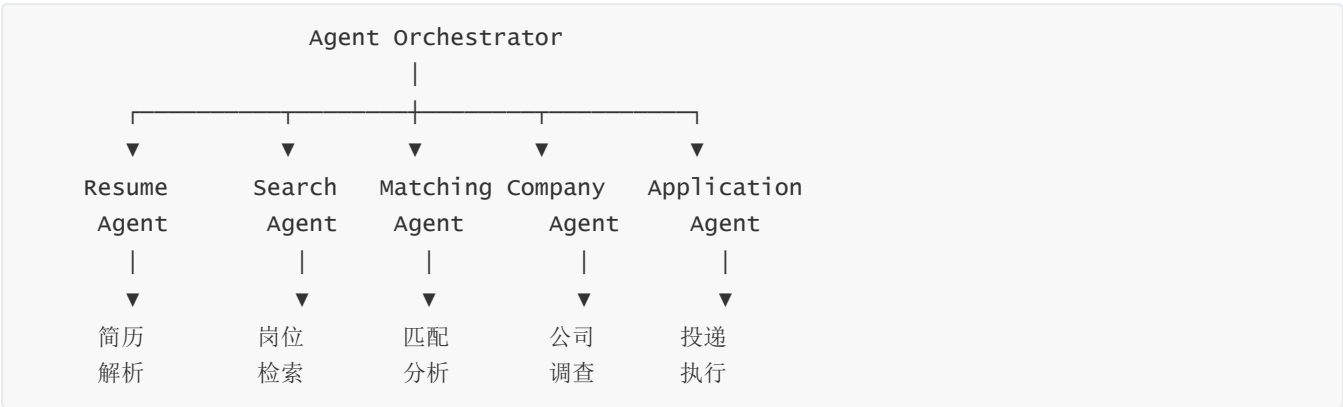
判断两个岗位是否可能为同一岗位。

去重后的岗位作为用户最终看到的推荐对象。

4. Agent 系统设计

4.1 Agent 总体架构

JobFlow 不采用单一 Agent 处理所有任务，而采用多个专业 Agent + Agent Orchestrator 的结构。



Agent Orchestrator 负责：

- Agent 调度；
- Agent 之间的数据传递；
- 工具调用；

- 任务状态控制;
- 异常处理;
- 结果汇总。

4.2 Resume Agent

输入

- 原始简历文件;
- 文本或图像解析结果。

输出

Candidate Profile

核心职责

- 提取个人基本信息;
- 提取教育经历;
- 提取项目与工作经历;
- 提取技能;
- 统一字段格式;
- 对缺失信息进行标记。

Resume Agent 不应主动编造简历中不存在的信息。

4.3 Job Search Agent

输入

- Candidate Profile;
- 用户求职偏好。

输出

Search Task

核心职责

- 理解用户求职目标;
- 生成搜索关键词;
- 分解搜索条件;
- 调用招聘平台 Adapter;

- 汇总岗位结果；
- 根据搜索结果进行必要的查询扩展。

例如用户输入：

“上海 Java 后端实习，8K 以上，最好互联网公司。”

Agent 可以拆解为：

职位：
Java / Java 后端 / 后端开发

城市：
上海

薪资：
>= 8K

工作类型：
实习

公司偏好：
互联网

4.4 Matching Agent

输入

Candidate Profile
+
Job
+
User Preference

输出

Match Result

包括：

- 综合匹配度；
- 技能匹配；
- 经历匹配；
- 学历匹配；
- 硬性条件匹配；
- 偏好匹配；
- 优势；

- 不足；
- 推荐等级；
- 推荐理由。

Matching Agent 的主要目标不是生成一个简单的相似度，而是完成可解释的岗位适配分析。

4.5 Company Agent

输入

Company

输出

Company Profile
+
Risk Information

核心职责

- 收集公开公司信息；
- 对公司信息进行结构化整理；
- 检查招聘主体与公司名称；
- 分析公开企业背景；
- 输出信息来源及风险提示。

该 Agent 输出的内容应以事实信息为基础，避免将无法验证的主观推断作为结论。

4.6 Application Agent

输入

Job
+
User Profile
+
Resume

输出

Application Result

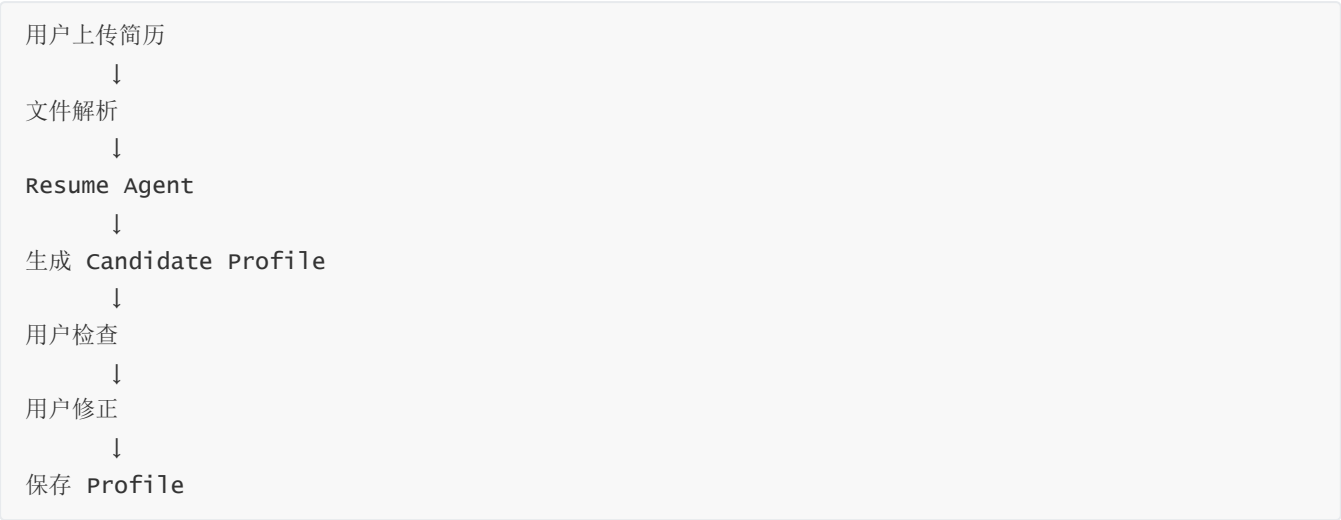
主要职责：

- 创建投递任务；

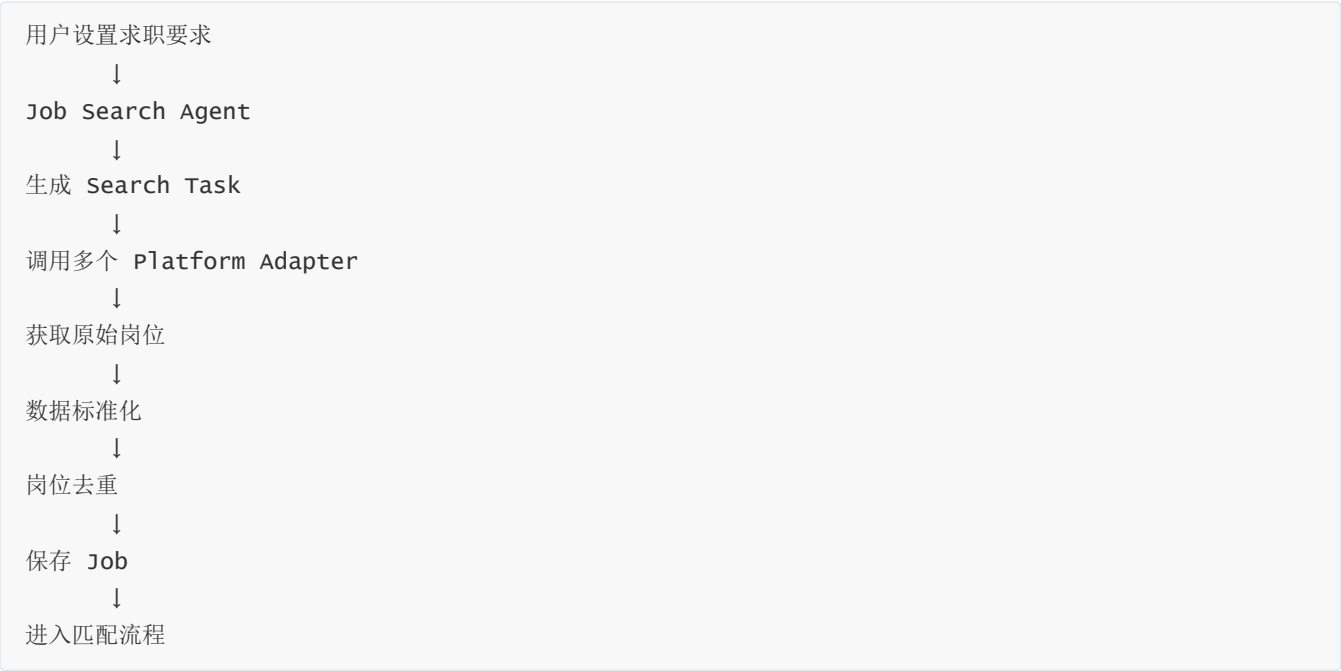
- 调用浏览器自动化工具；
- 执行页面操作；
- 监控操作结果；
- 保存投递状态；
- 处理可恢复异常；
- 在必要时暂停并请求用户接管。

5. 核心业务流程设计

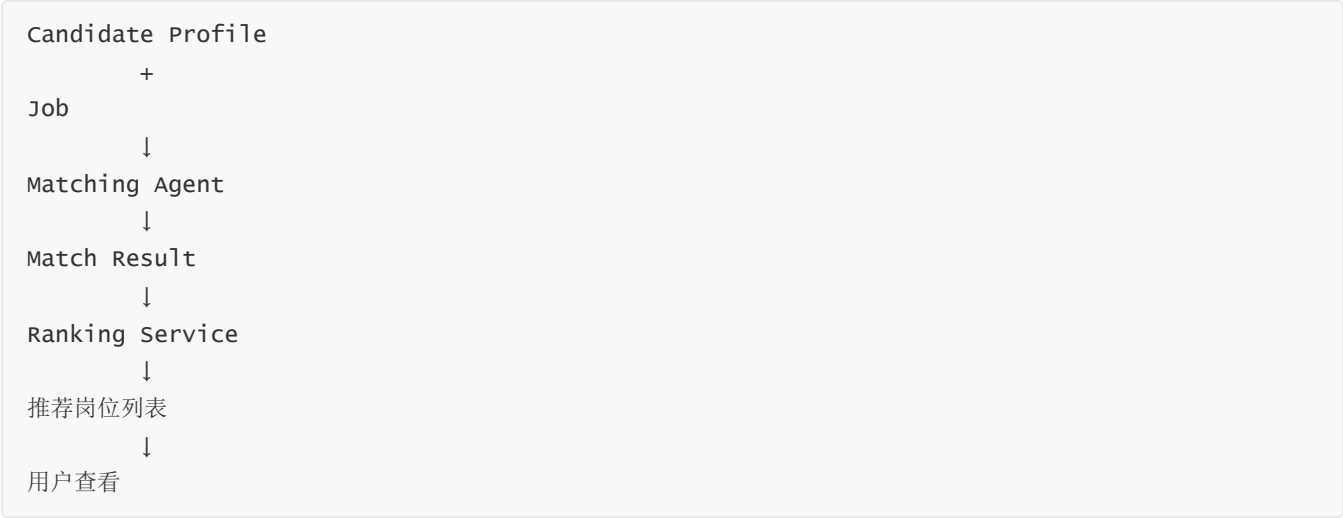
5.1 用户画像建立流程



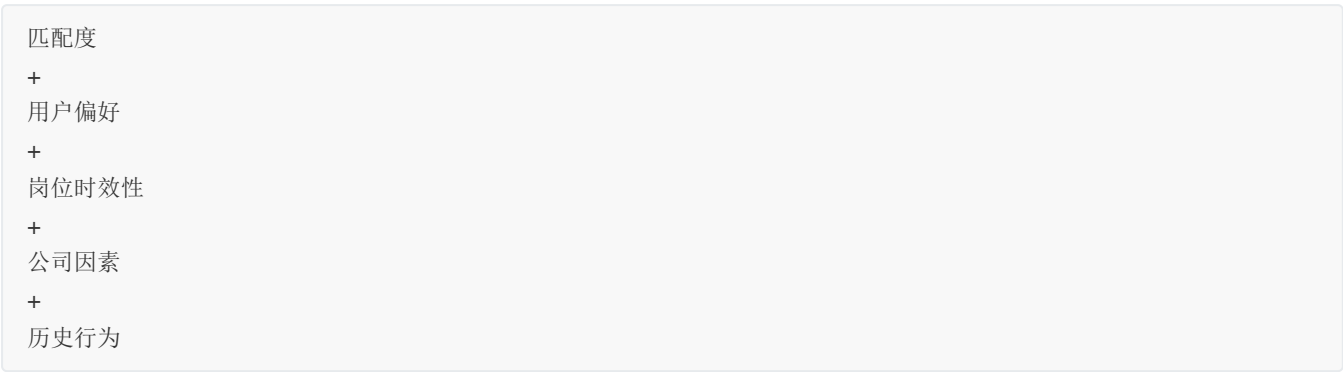
5.2 岗位搜索流程



5.3 岗位匹配与推荐流程

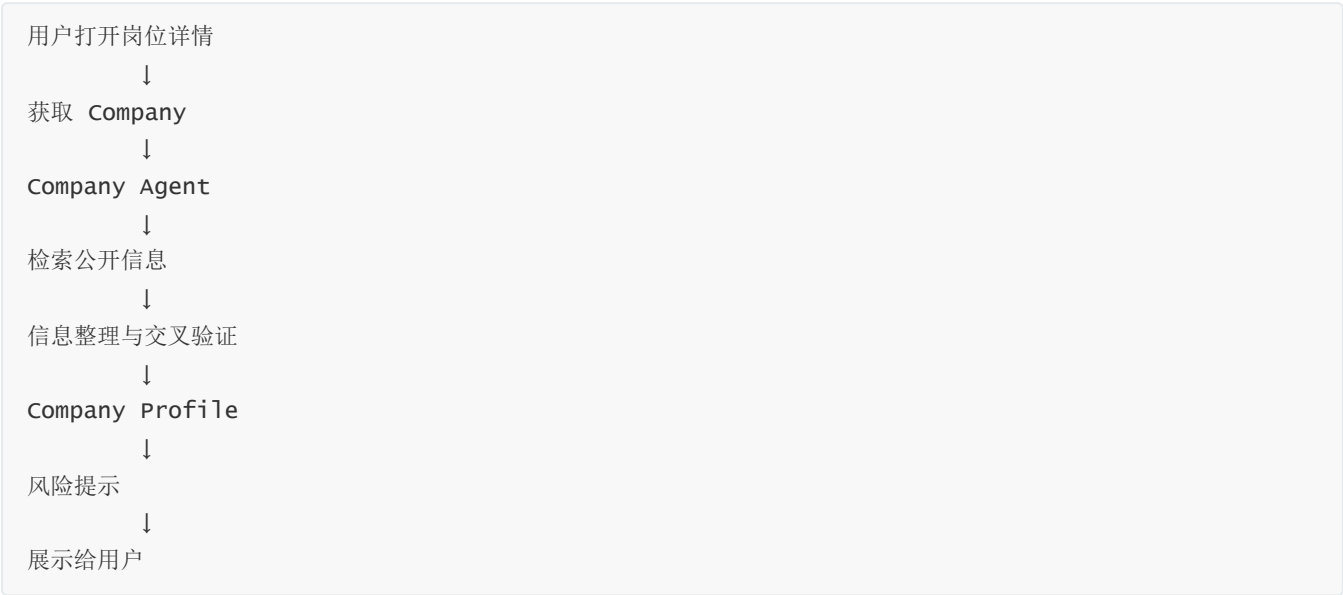


Ranking Service 可以综合：

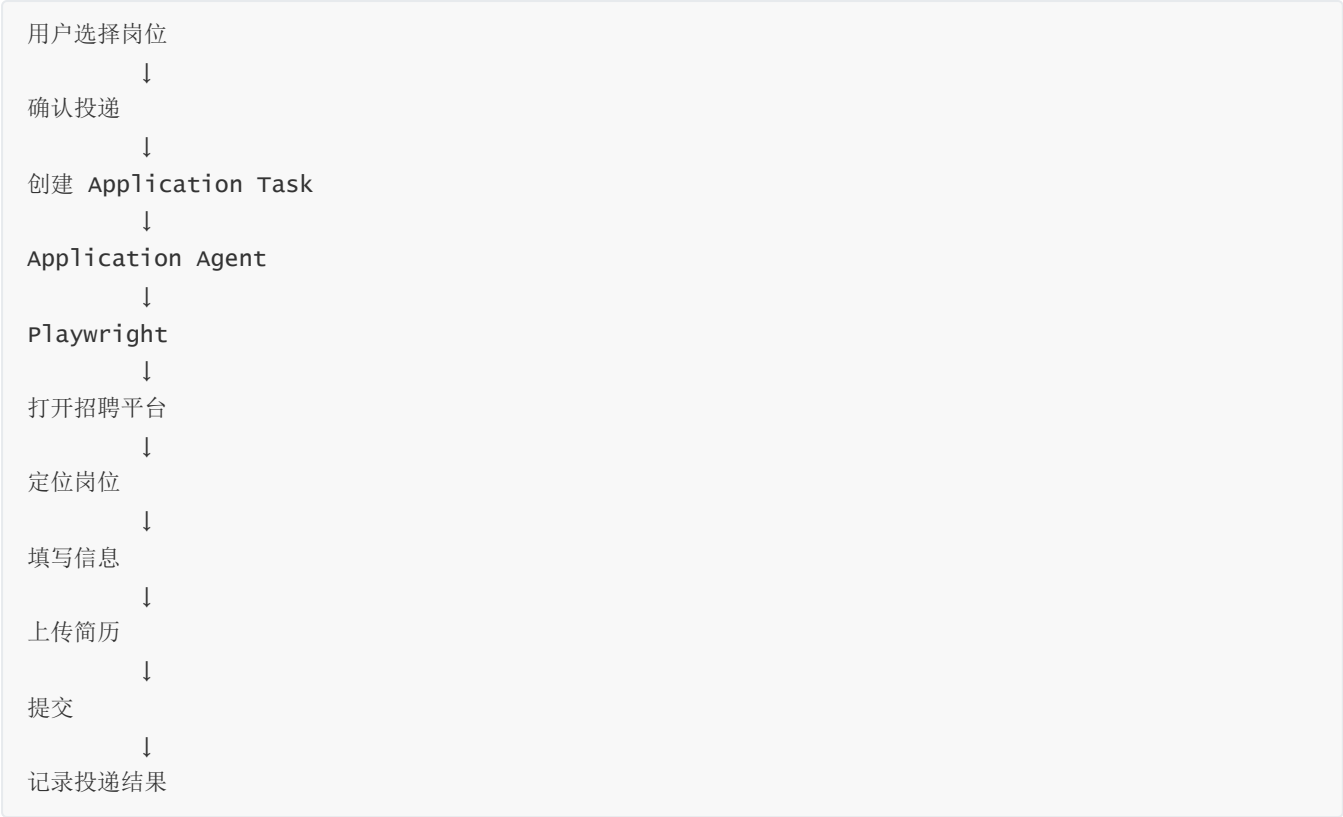


对岗位进行最终排序。

5.4 公司分析流程



5.5 投递流程



6. 数据架构设计

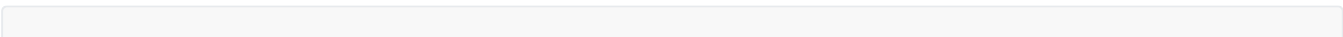
6.1 核心实体

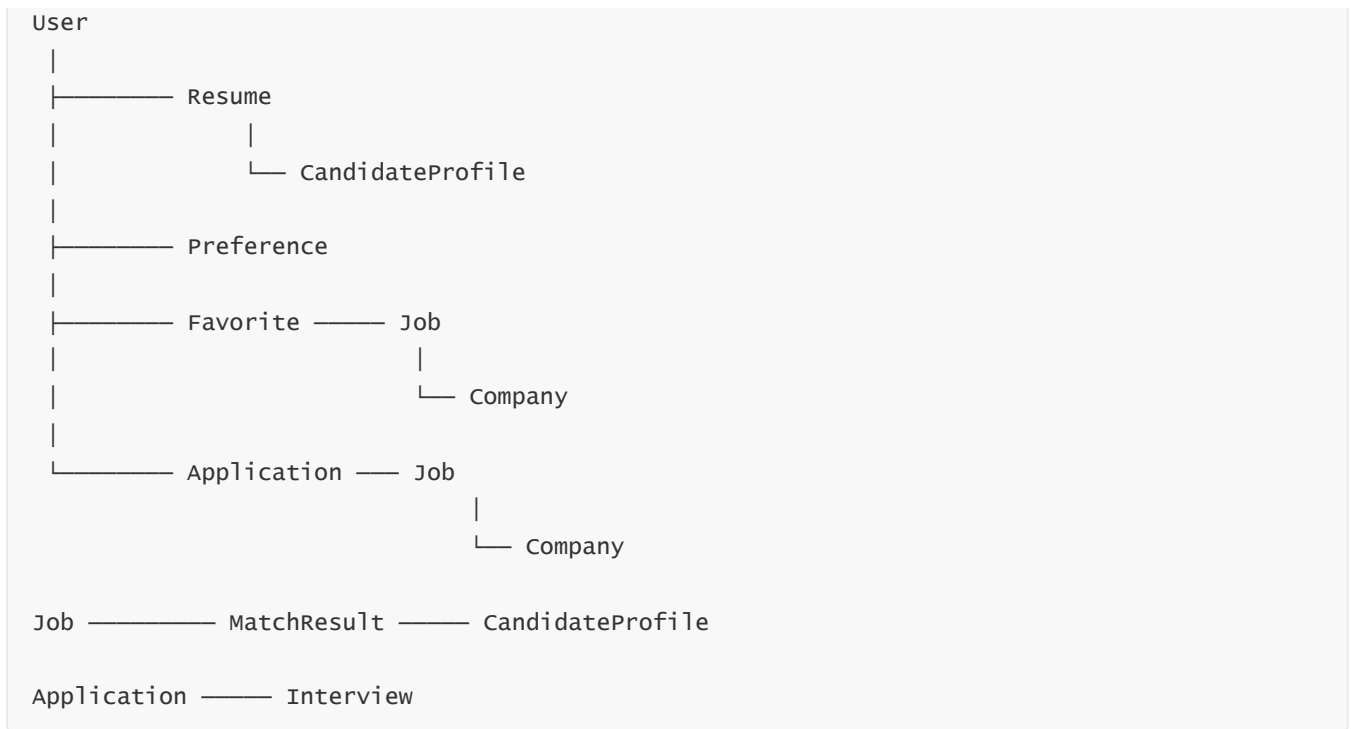
系统核心实体包括：

- User
- Resume
- CandidateProfile
- Preference
- Job
- Company
- JobSource
- MatchResult
- Favorite
- Application
- Interview
- AgentTask
- AgentLog

6.2 核心实体关系

核心数据关系如下：





其中 Job 是系统中的核心业务对象之一，用户的收藏、匹配和投递等行为均围绕 Job 展开。

6.3 数据存储设计

系统可以采用关系型数据库保存核心业务数据。

MySQL 主要保存：

- 用户；
- 简历；
- 用户画像；
- 岗位；
- 公司；
- 匹配结果；
- 收藏；
- 投递；
- 面试记录。

Redis 主要用于：

- 缓存；
- Agent 临时状态；
- 异步任务状态；
- 分布式锁；
- 短期数据。

简历原始文件可以采用本地文件存储或对象存储，并在数据库中保存文件元信息。

7. 任务与状态管理

7.1 异步任务

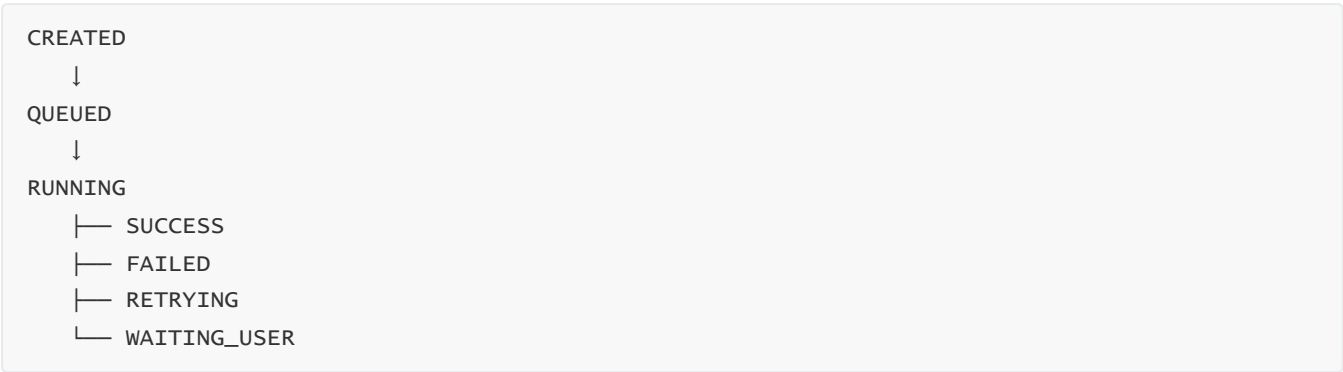
以下操作设计为异步任务：

- 简历解析；
- 多平台岗位搜索；
- 岗位批量分析；
- 公司信息调查；
- 自动投递；
- 定时岗位刷新。

系统使用 Celery 负责任务执行，Redis 负责消息队列和任务状态管理。

7.2 Agent 任务状态

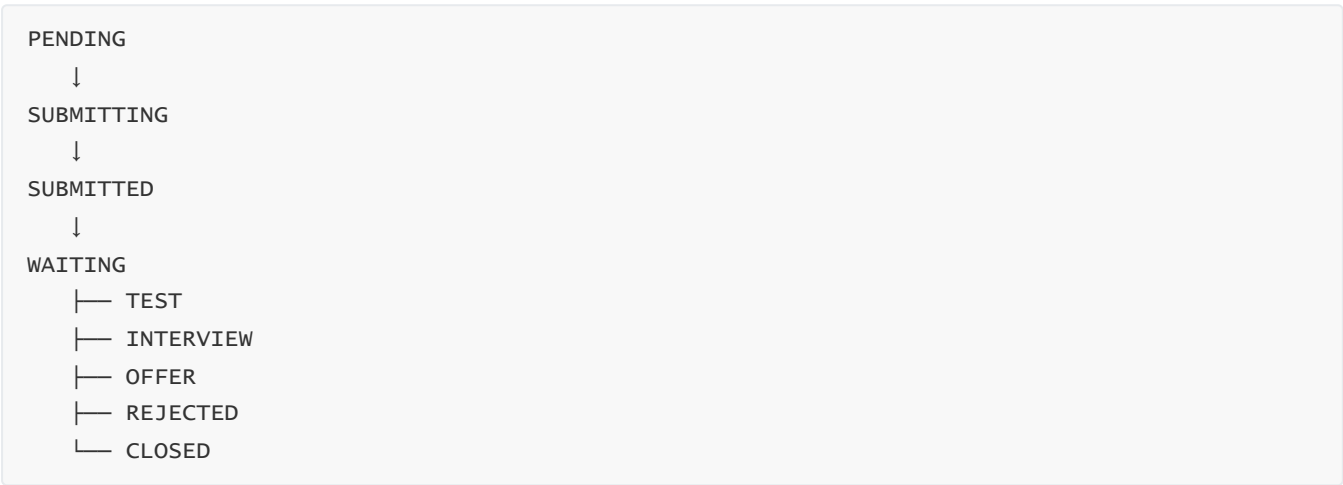
每一个 Agent Task 均具有明确状态：



用户可以在桌面端查看任务状态。

7.3 投递状态

投递业务状态可以设计为：



用户也可以手动修改岗位状态，以适应招聘平台无法自动获取最新状态的情况。

8. 异常处理设计

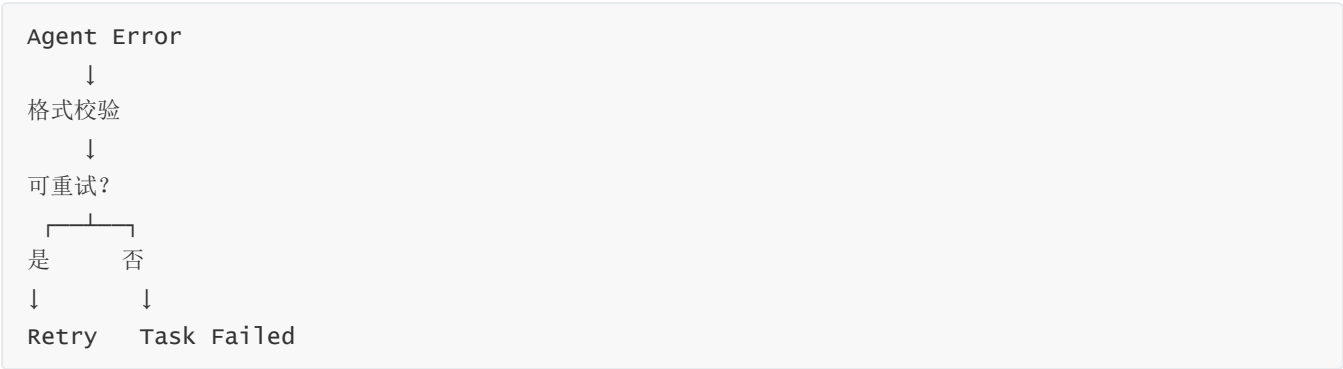
系统针对 Agent 和浏览器自动化可能出现的异常建立统一处理机制。

8.1 Agent 异常

包括：

- LLM 请求失败；
- 输出格式错误；
- JSON 解析失败；
- 信息不足；
- 工具调用失败。

处理方式：

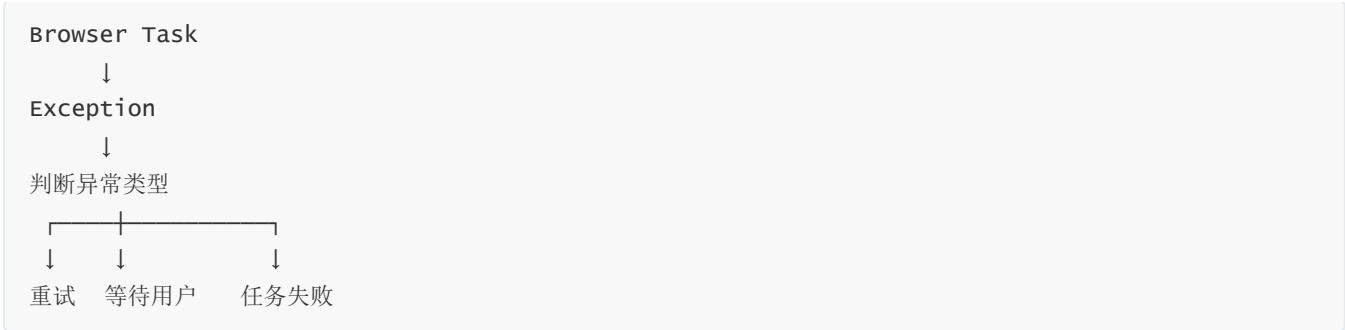


8.2 浏览器自动化异常

包括：

- 网络异常；
- 页面加载失败；
- 登录失效；
- 页面结构变化；
- 元素不存在；
- 验证码；
- 操作结果无法确认。

处理方式：



对于验证码或需要用户判断的场景，任务进入 `WAITING_USER` 状态。

用户完成操作后，Agent 可以继续执行后续步骤。

9. 外部系统设计

9.1 招聘平台

系统通过 Platform Adapter 对接招聘平台。

每个平台 Adapter 对外提供统一接口：

```
search_jobs()
get_job_detail()
get_company_info()
```

核心服务只依赖统一接口，不直接依赖具体平台实现。

9.2 大语言模型

Agent 层通过统一 LLM Service 访问大语言模型。

初期支持：

- 通义千问；
- GLM-4。

后续可以增加其他模型。

Agent 不直接绑定某一个模型，而通过统一的 LLM 接口进行调用。

9.3 浏览器自动化

Playwright 作为 Browser Tool，为 Application Agent 提供：

- 浏览器启动；
- 页面访问；
- 元素定位；
- 表单填写；

- 文件上传；
- 页面截图；
- 操作结果获取。

Playwright 不负责判断“应该投递什么岗位”，只负责执行 Agent 已经确定的浏览器操作。

10. 安全与隐私设计

由于 JobFlow 涉及完整个人简历和招聘平台登录状态，系统需要重点保护用户隐私。

主要措施包括：

1. 简历数据进行权限隔离；
2. 敏感信息不直接写入普通日志；
3. API Key 不硬编码；
4. 招聘平台登录信息采用安全方式保存；
5. Agent 日志进行脱敏处理；
6. 用户能够删除简历及相关数据；
7. 自动投递操作必须在用户授权范围内执行。

11. 可扩展性设计

11.1 招聘平台扩展

采用 Adapter 机制：

```
JobSource
|
├─ Adapter A
├─ Adapter B
├─ Adapter C
└─ Adapter D
```

新增平台时只需要实现对应 Adapter。

11.2 Agent 扩展

通过统一 Agent Interface 管理不同 Agent。

例如：

BaseAgent

|

├ ResumeAgent

├ JobSearchAgent

├ MatchingAgent

├ CompanyAgent

└ ApplicationAgent

后续可以增加：

ResumeOptimizationAgent

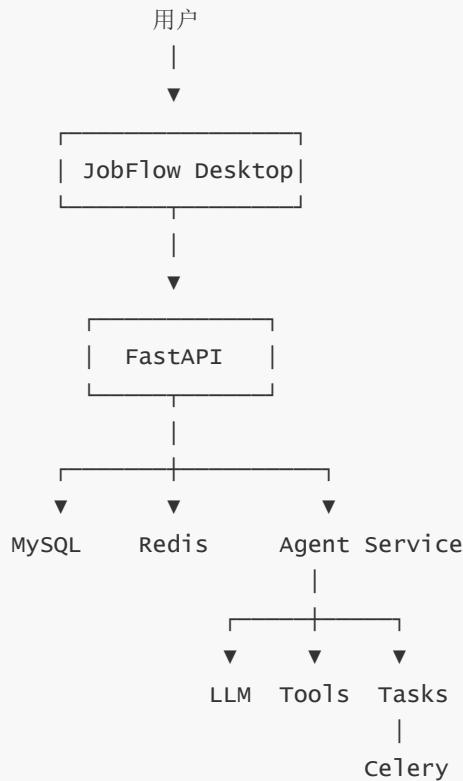
InterviewPreparationAgent

CareerPlanningAgent

而无需修改系统整体架构。

12. 系统部署结构

初步部署结构如下：



系统可以通过 Docker 对后端服务进行容器化部署。

13. 概要设计总结

JobFlow 的整体设计采用**桌面端 + 业务服务 + Agent + 工具适配层 + 数据层**的分层架构。

在业务层面，以岗位作为核心业务对象，形成：

用户画像 → 岗位发现 → 岗位匹配 → 推荐排序 → 公司调查 → 用户决策 → 投递 → 跟踪

的完整业务闭环。

在智能层面，通过多个专业 Agent 分工处理不同任务：

Resume Agent 负责理解用户，Job Search Agent 负责发现岗位，Matching Agent 负责评估岗位，Company Agent 负责调查企业，Application Agent 负责执行投递。

在工程层面，通过 Adapter 机制解决多平台接入问题，通过 Redis + Celery 解决耗时任务和异步执行问题，通过状态机和 Human-in-the-loop 机制解决自动化过程中的异常和不确定性。

该设计为后续的数据库详细设计、API 设计、UI 设计和具体 Agent 实现提供统一的系统架构基础。